

The Arzens

cybersecurity foundations and Professional Readiness

Security Testing Report

Submitted By:

Name: Mahnoor Shuaib

Reference Code: ARZ-2026-YJJW

Internship Track: Cyber Security foundations and Professional Readiness

Finding 1:

Issue in Ambassador Registration Form

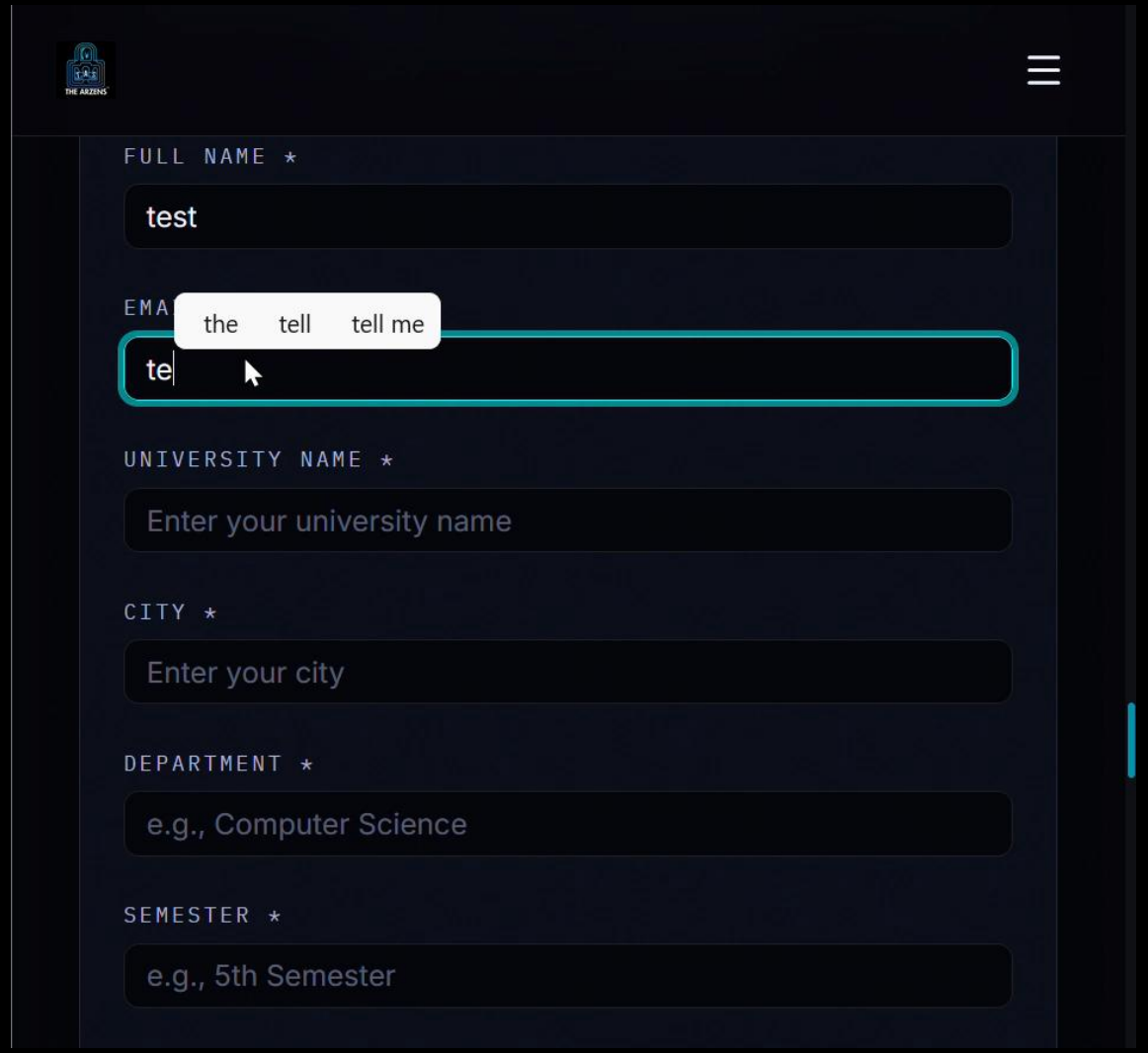
Issue Title: Improper Input Validation – Ambassador Registration Form (Multiple Fields)

Affected Page/Feature: Ambassador Registration Form (University Name, City, Department, Semester fields)

Steps to Reproduce:

- Navigate to the Ambassador registration form on the website.
- Fill in the Name and Email fields with valid data.
- In the **University Name** field, enter numeric characters instead of a valid university name (e.g., 12345).
- In the **City** field, enter numeric characters instead of a valid city name (e.g., 12345).
- In the **Department** field, enter numeric characters instead of a valid department name (e.g., 12345).
- In the **Semester** field, enter an invalid value such as a negative number (e.g., -5) or an out-of-range number (e.g., 12345).
- Submit the form.

Video recording:



Expected Result: The form should validate each field according to its expected data type (e.g., text-only for University Name, City, and Department; a valid numeric range such as 1–8 for Semester) and reject the submission with an appropriate error message if invalid data is entered.

Actual Result: The form accepted numeric input in all text-based fields (University Name, City, Department) and accepted illogical/out-of-range values in the Semester field (e.g., negative numbers, out-of-range numbers).

The application was submitted successfully in every case without any client-side or server-side validation error.

Severity/Impact: Low (🔵)

Allows storage of illogical and inconsistent data in the backend database, affecting data integrity. Indicates a likely absence of server-side input validation across the form. Could allow automated bots or malicious users to flood the database with junk/fake registrations.

Recommended Solution:

- Implement server-side input validation for all form fields, not just client-side (JavaScript) checks, since client-side validation can be bypassed.
- Restrict the **Semester** field to a defined numeric range (e.g., dropdown selection of 1–8, or numeric min/max validation).
- Restrict **University Name**, **City**, and **Department** fields to alphabetic/text input patterns using regex validation, disallowing purely numeric entries.
- Apply consistent validation rules across the entire form and return clear error messages when invalid data is submitted.

Finding 2:

Issue Title: Unrestricted File Upload

No File Extension/Type Validation on Resume Field

Finding 2:

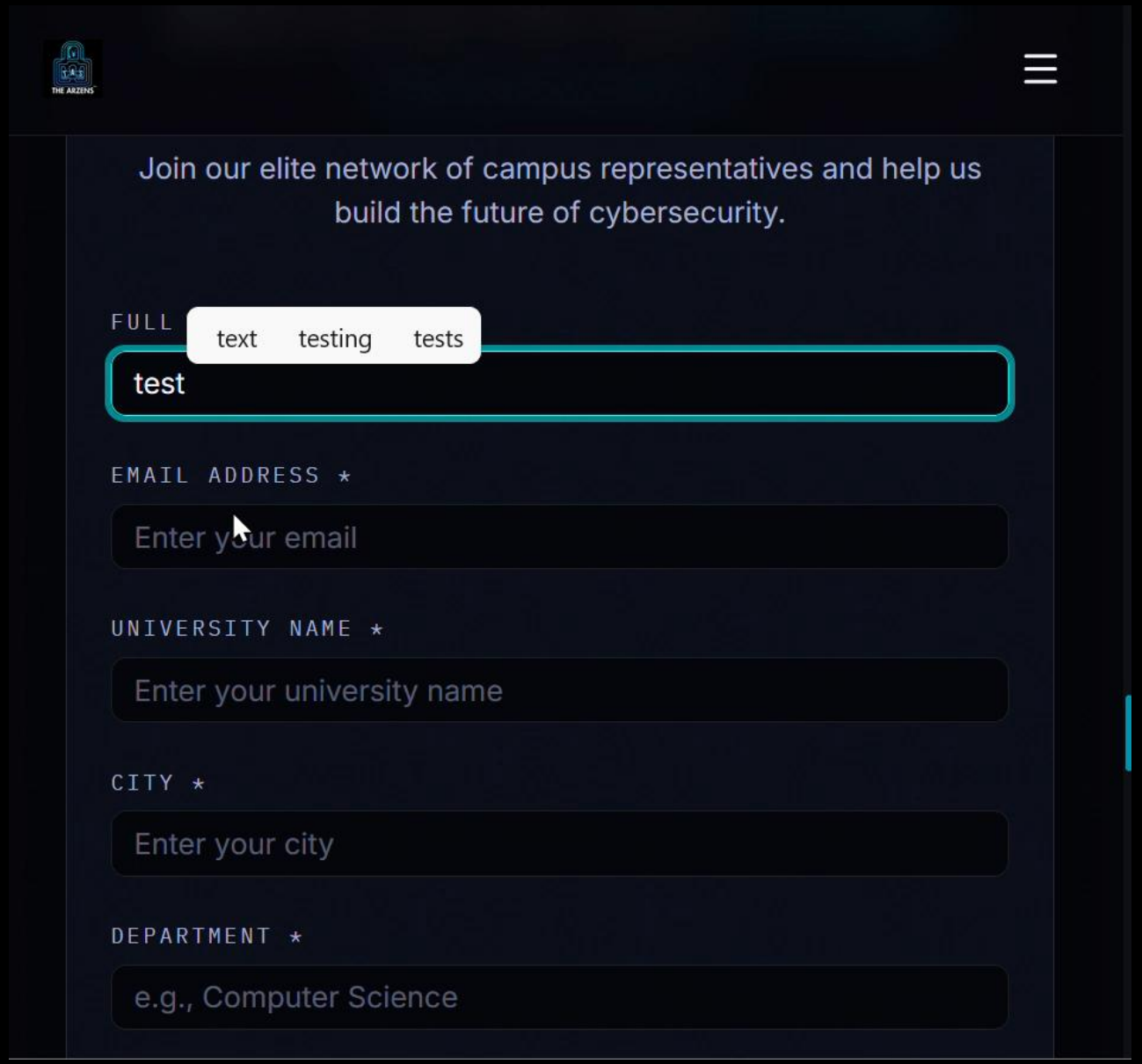
Unrestricted File Upload, No File Extension/Type Validation on Resume Field

Affected Page/Feature: Ambassador Registration Form (Resume Upload field)

Steps to Reproduce:

- Navigate to the Ambassador registration form.
- Fill all required fields with valid data.
- In the **Resume upload** field, select and upload a file with a non-document extension (e.g., a .py Python file) instead of the expected .pdf/.doc format.
- Submit the form.

Video Recording:



Expected Result: The system should validate the uploaded file's type/extension and reject any file that is not an approved document format (e.g., PDF, DOC, DOCX), returning a clear error message

Actual Result: The system accepted a .py (Python script) file in the Resume upload field without any restriction, and the application was submitted successfully.

Severity/Impact: Medium (🔵)

- Lack of file type/extension validation on file uploads is a significant risk area: if the server does not also validate file content and storage handling, it could potentially allow upload of executable or script files, which depending on server configuration, may lead to remote code execution, server compromise, or defacement.
- Even without further exploitation, this represents a meaningful gap in input validation that goes beyond simple data-integrity concerns, since uploaded files are typically stored and may be accessed/executed under certain server misconfigurations.

Recommended Solution:

- Validate uploaded file types both by **file extension** and **MIME type/file signature (magic bytes)** — do not rely on extension alone, as it can be spoofed.
- Restrict accepted file types strictly to expected formats (e.g., .pdf, .doc, .docx).
- Store uploaded files outside the web root, or in a location where they cannot be directly executed by the server.
- Rename uploaded files upon storage (e.g., using a generated ID) rather than preserving user-supplied file names.
- Implement a maximum file size limit and scan uploaded files for malware where feasible.